



# Analytics & Big Data

Session 5: Regression II

Prof. Dr. Gerit Wagner

(2026-04-14)

## Learning objectives

- **Explain** how business problems can be formulated as binary classification tasks and modeled using logistic regression.
- **Understand** how logistic regression uses a linear predictor and sigmoid function to produce probabilities, apply thresholds, and interpret log-odds and coefficients.
- **Evaluate** classification models using confusion matrices and metrics such as accuracy, precision, recall, and F1 score.
- **Apply** model predictions to decision-making by selecting appropriate thresholds based on business costs and expected value.

# Business problem: Churn prediction

# Predicting customer churn

A lost customer is often a predictable customer. Firms collect many traces of customer relationships — usage, spending, support interactions, contract data. The analytical challenge is to turn these traces into an early warning system.

## Typical variables in a churn dataset

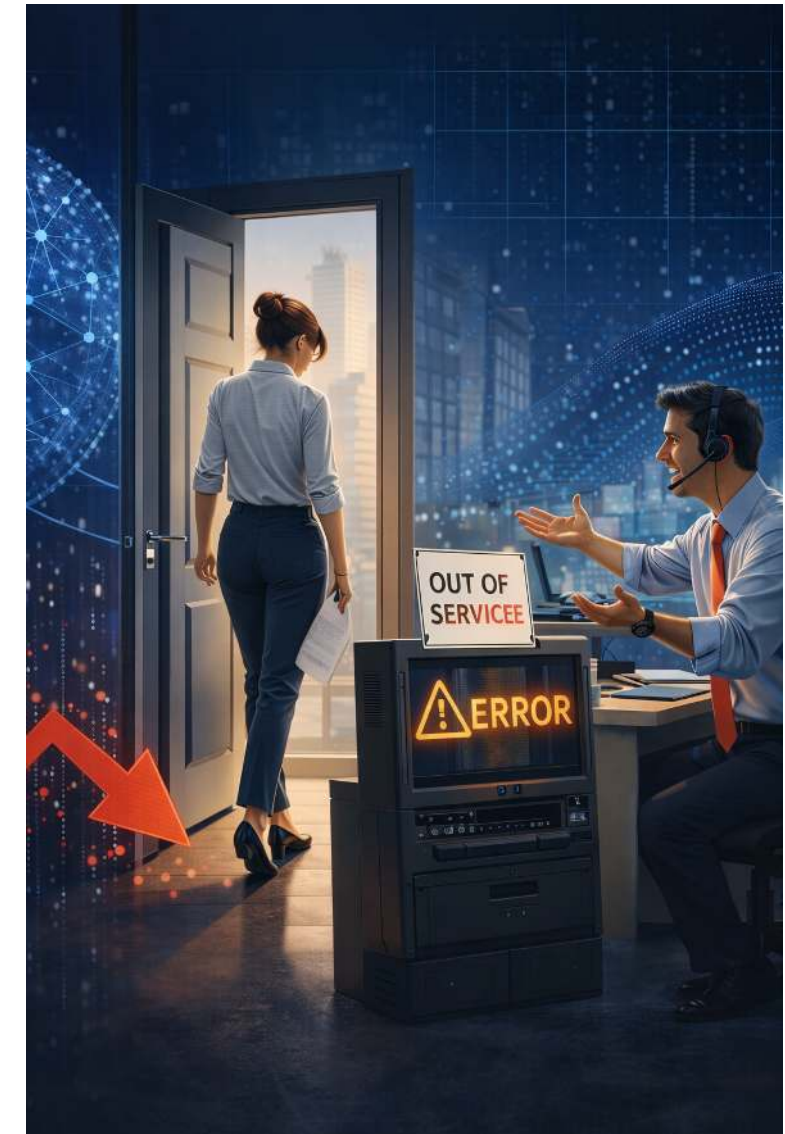
- **Who is the customer?** tenure, segment, contract type, subscription tier
- **How do they use the service?** login frequency, feature usage, inactivity days
- **How satisfied are they?** complaints, support tickets, response times, NPS/satisfaction
- **Are there warning signs?** payment delays, downgrades, cancellations of add-ons, price sensitivity

## Prediction goal

Predict:

$$P(\text{churn} = 1 \mid X)$$

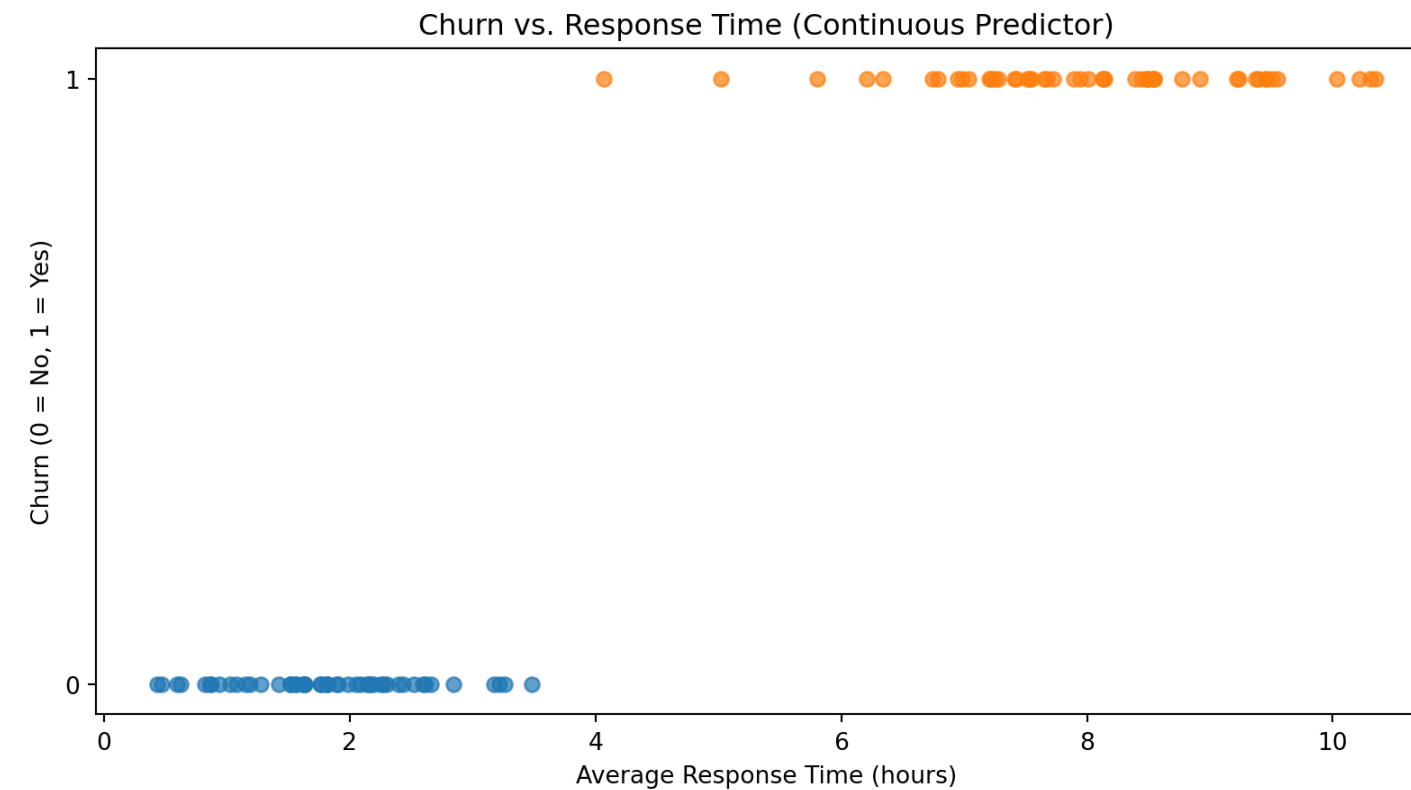
where  $X$  contains the observed customer characteristics and behaviors.



# Geometric view of the problem



Churn prediction is a **binary classification problem**:



Possible approach: We could use Linear Regression with a threshold at 0.5 to classify output:

- If  $f(x) < 0.5$ , predict  $y=0$
- If  $f(x) \geq 0.5$ , predict  $y=1$

# Logistic regression

# Classification



Examples:

- Online transactions: Fraudulent (yes/no)?
- Customer churn: yes/no?
- EMail: Spam/not spam?

$$y \in \{0, 1\}$$

with

0: “Negative class” (e.g., no fraud, no churn, no spam)

1: “Positive class” (e.g., fraud, churn, spam)



# Logistic regression model

Ideally, our model should predict the **probability** of  $y_i = 1$ . This would allow us to apply a threshold for classification, but it would also give us more information (how likely an observation belongs to the *positive class*).

This means we need a model that produces predicted probabilities between 0 and 1:

## Step 1: Linear model

$$z_i = \beta_0 + \beta_1 x_i$$

## Step 2: Transform to probability via sigmoid function

$$\mathbb{P}(y_i = 1 \mid x_i) = \sigma(z_i) \text{ with } \sigma(z) = \frac{1}{1 + e^{-z}}$$

## Logistic regression model

$$\mathbb{P}(y_i = 1 \mid x_i) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_i)}}$$

### Key intuition:

- Linear predictor  $z \rightarrow$  transformed by the sigmoid (logistic) function
- Output always in  $[0, 1]$
- Can be interpreted as a probability (next slide)



# Log-odds

Logistic regression assumes that the **log-odds of the outcome are linear in the predictors**:

$$\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 x_i$$

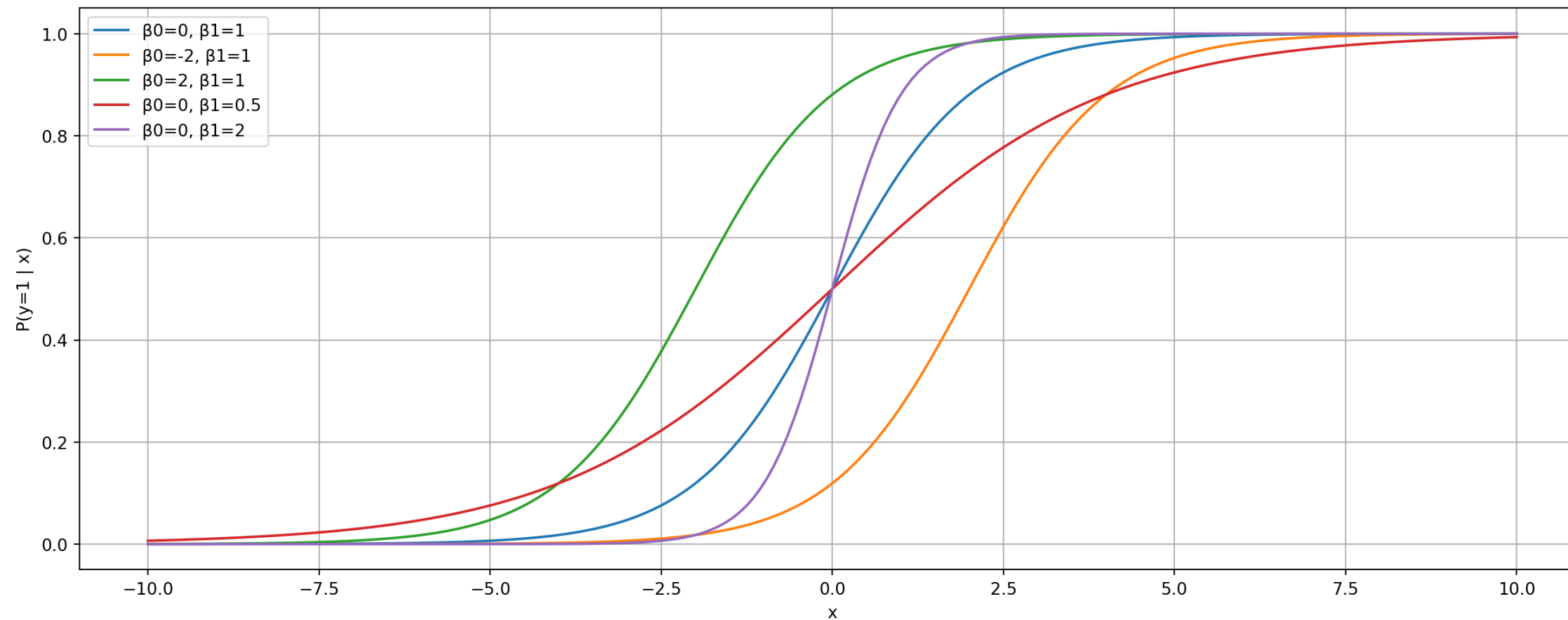
Note:  $\frac{p_i}{1-p_i}$  are the “odds”.

This implies a **sigmoid-shaped relationship** between  $x_i$  and the probability of  $y_i = 1$ :

$$p_i = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_i)}}$$

The sigmoid function serves as a *link function* to convert the linear predictor into a *probability*.

# The Sigmoid function



## *i* Note

- Changing  $\beta_0$  shifts the curve left/right
- Changing  $\beta_1$  changes slope (flatter vs. steeper transition)

# Estimation: Maximum likelihood

Instead of minimizing squared errors, logistic regression chooses parameters that make the observed data most likely.

**Core idea:** We want predicted probabilities to match observed outcomes:

- If  $y_i = 1 \rightarrow$  predicted probability should be **high**
- If  $y_i = 0 \rightarrow$  predicted probability should be **low**

## Likelihood function

For each observation:  $P(y_i | x_i) = p_i^{y_i} (1 - p_i)^{1-y_i}$

For all observations:

$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}$$

**Maximization:** Choose  $\beta_0, \beta_1$  to maximize this likelihood

💡 **Intuition:** We search for the model that assigns high probability to what actually happened.

No need to memorize the formula. Understand the idea.

# Logistic regression in Python

We estimate the model using the sklearn library:

```

1 import pandas as pd
2 from sklearn.linear_model import LogisticRegression
3
4 df = pd.DataFrame({
5     "response_time": [1, 2, 3, 5, 7, 8, 9, 11],
6     "churn":          [0, 0, 0, 0, 1, 1, 1, 1]
7 })
8 X = df[["response_time"]]
9 y = df["churn"]
10
11 model = LogisticRegression()
12 model.fit(X, y)
13
14 print("Intercept ( $\beta_0$ ):", model.intercept_[0])
15 print("Slope ( $\beta_1$ ):", model.coef_[0][0])

```

```

Intercept ( $\beta_0$ ): -6.201807152628621
Slope ( $\beta_1$ ): 1.0606214874843112

```

# Interpretation of coefficients

Logistic regression coefficients are not directly effects on probability. They describe effects on the log-odds.

## Coefficient meaning

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x$$

- $\beta_1$ : change in **log-odds** when  $x$  increases by 1

### More intuitive: odds ratios

Exponentiate the coefficient:  $e^{\beta_1}$

- Factor change in the **odds**
- Example:
  - $e^{\beta_1} = 1.5 \rightarrow$  odds increase by **50%**
  - $e^{\beta_1} = 0.7 \rightarrow$  odds decrease by **30%**

### Important implication

- Effect on probability is **nonlinear**
- Depends on current level of  $x$



# Evaluation

# Predicted probabilities

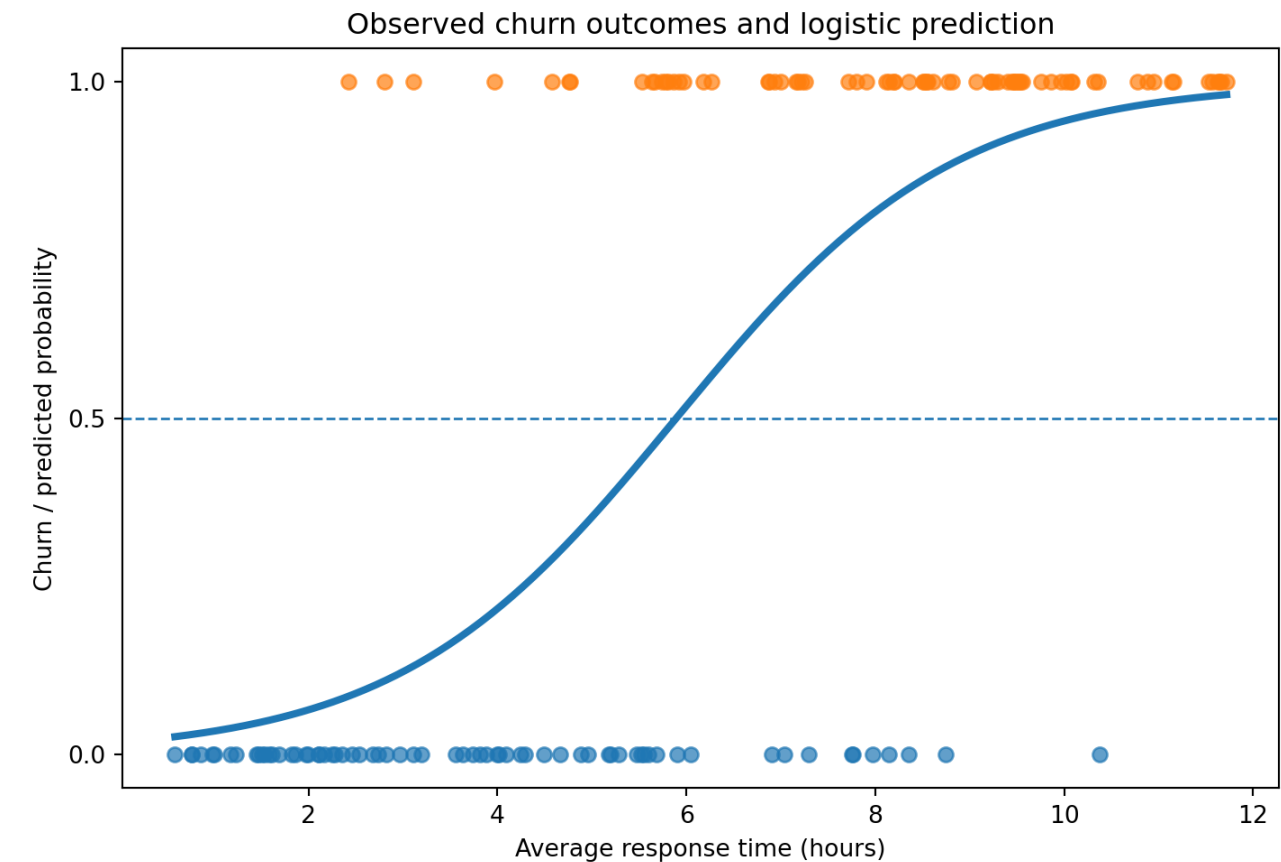


A logistic regression model predicts a **probability** of churn:

$$P(\text{churn}_i = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_i)}}$$

A threshold (such as **0.5**) is needed to convert probabilities into class labels.

Comparing predicted vs. actual class labels allows us to evaluate the performance of a classifier like logistic regression.



Note: there are no meaningful *residuals* (or  $R^2$ ), as in linear regression models.



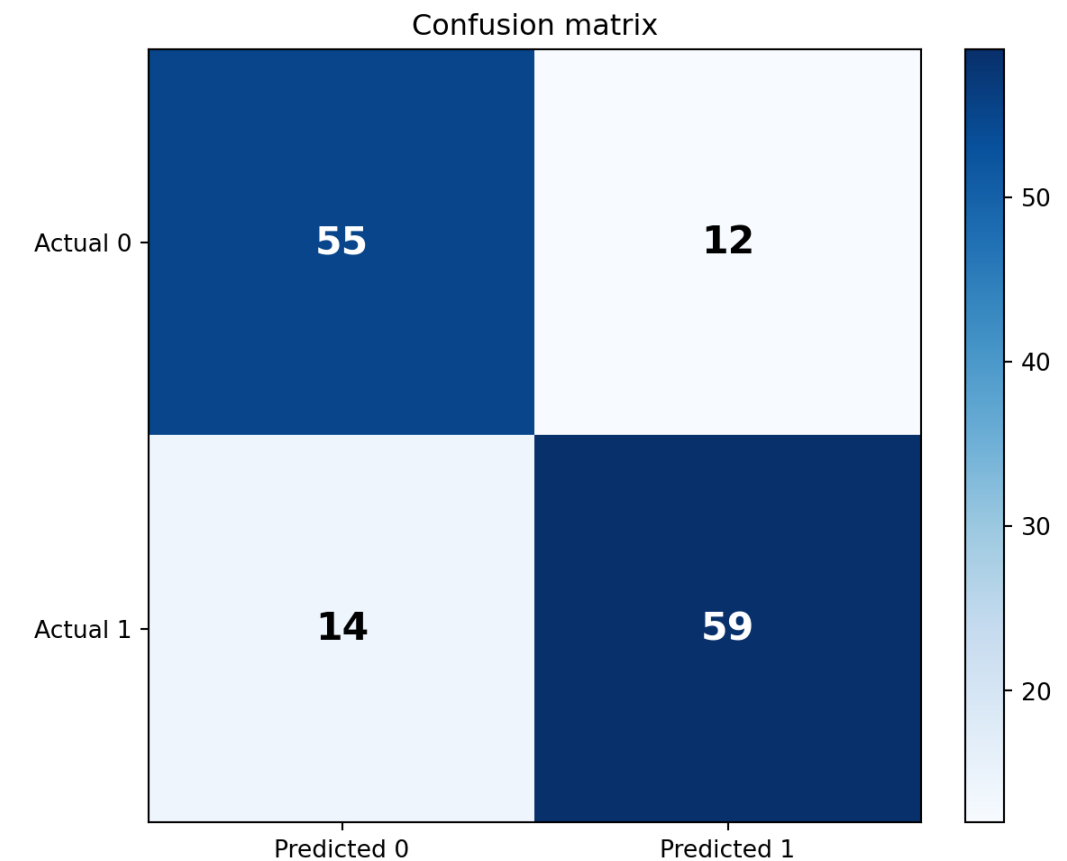
# Confusion matrix



Using a **0.5 threshold**, predicted probabilities are turned into classes.

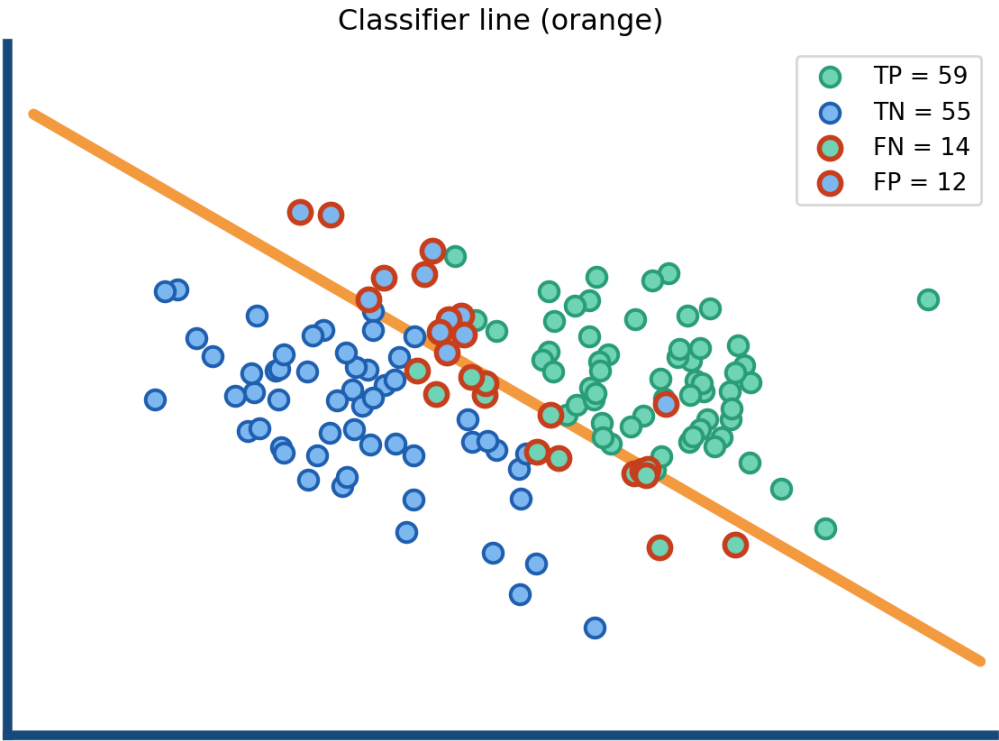
The confusion matrix summarizes:

- **True negatives (TN):** predicted no churn, observed no churn
- **False positives (FP):** predicted churn, observed no churn
- **False negatives (FN):** predicted no churn, observed churn
- **True positives (TP):** predicted churn, observed churn





# Evaluation metrics



| Metric    | Formula                                                                                 | Value | Interpretation                                             |
|-----------|-----------------------------------------------------------------------------------------|-------|------------------------------------------------------------|
| Accuracy  | $\frac{TP+TN}{TP+TN+FP+FN}$                                                             | 81.4% | Overall share of correct predictions                       |
| Precision | $\frac{TP}{TP+FP}$                                                                      | 83.1% | Among predicted positives, how many are actually positive? |
| Recall    | $\frac{TP}{TP+FN}$                                                                      | 80.8% | Among actual positives, how many did the model identify?   |
| F1        | $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ | 81.9% | Balance between precision and recall                       |

# ROC curve and AUC



**ROC** (Receiver Operating Characteristic) curve summarizes the trade-off between:

- **True Positive Rate** (aka. recall or sensitivity)
- **False Positive Rate**

As we vary the **classification threshold**, the model becomes more or less conservative in predicting **churn = 1**.

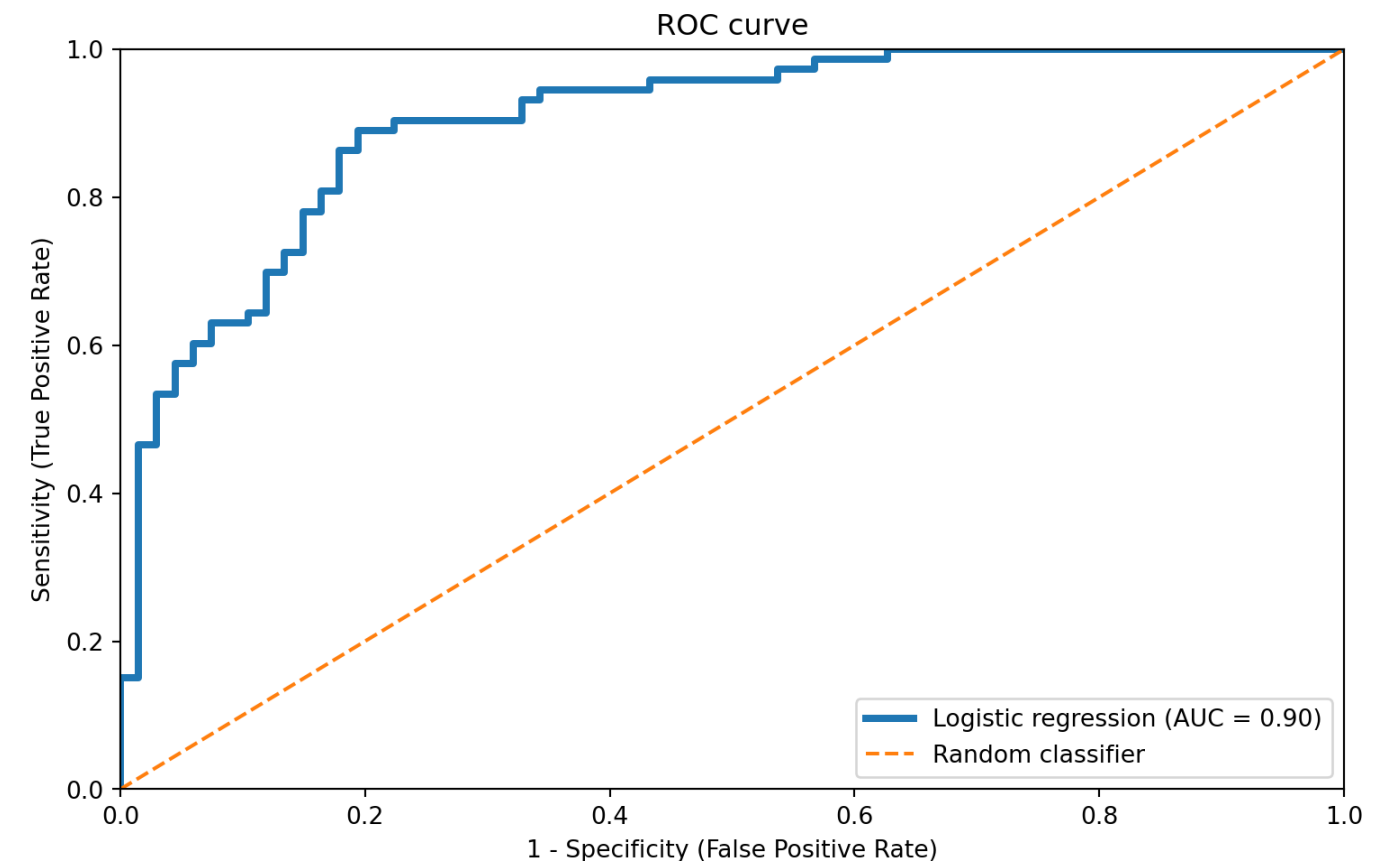
**Area under the curve (AUC)** summarizes ROC performance in a single number:

- **AUC = 0.5**: no better than random guessing
- **AUC = 1.0**: perfect separation
- **Higher AUC**: better ranking of churners vs. non-churners

A good classifier achieves:

- **high true positive rate**
- **low false positive rate**

→ curves closer to the **top-left corner** indicate better model fit.



# From prediction to decisions

# From prediction to decision

A logistic regression model predicts a **probability of churn**:

$$p(x) = P(\text{churn} = 1 \mid x)$$

This probability is based on historical data, reflecting how similar customers behaved in the past, and capturing a baseline churn risk. It is not a certainty.

Suppose a firm has two possible actions:

- **Intervene:** offer a retention incentive
- **Do nothing**

So the business problem is no longer only:

Who is likely to churn?

It becomes:

For whom is an intervention worth it?

# Decision-making under uncertainty



We can use the expected value criterion to **translate predictions into decisions** by combining:

- **Predicted churn probability** from the model
- **Business consequences** of the action

A simple payoff structure:

| Action     | Customer would churn | Customer would stay |
|------------|----------------------|---------------------|
| Intervene  | +20                  | -10                 |
| Do nothing | -100                 | 0                   |

Interpretation:

- Saving a likely churner creates value
- Intervening with a customer who would stay creates unnecessary cost
- Missing a true churner is very costly

# Expected value of (not) intervening

| Action     | Customer would churn | Customer would stay |
|------------|----------------------|---------------------|
| Intervene  | +20                  | -10                 |
| Do nothing | -100                 | 0                   |

Probabilities:

- With probability  $p(x)$ , the customer is at risk and intervention helps
- With probability  $1 - p(x)$ , the customer would stay anyway and intervention wastes resources

If we **intervene**, the expected value is:

$$EV(intervene) = p(x) \cdot 20 + (1 - p(x)) \cdot (-10) = 30p(x) - 10$$

If we **do nothing**, the expected value is:

$$EV(do\_nothing) = p(x) \cdot (-100) + (1 - p(x)) \cdot 0 = -100p(x)$$

Based on Elkan ([2001](#)).

# Optimal decision rule



Instead of selecting one action, we should intervene whenever:

$$EV(intervene) > EV(do\_nothing)$$

Insert the expected values:

$$p(x) \cdot 20 + (1 - p(x)) \cdot (-10) > -100p(x)$$

Simplify:

$$20p(x) - 10 + 10p(x) > -100p(x)$$

$$130p(x) > 10$$

$$p(x) > 0.077$$



# Decision threshold

This gives a clear rule:

Intervene if  $p(x) > 0.077$  (if predicted churn probability exceeds 7.7%).

The threshold is low because:

- Losing a customer is **very costly**
- Intervening is **relatively cheap**

So even a small churn probability can justify action.

💡 A classification threshold does not have to be 0.5. It should depend on the **business context**.



# Two sources of knowledge

This decision combines two complementary inputs:

## 1. Data-driven component

- Logistic regression estimates:

$$p(x) = P(\text{churn} = 1 \mid x)$$

- Learned from historical data
- Provides a probability (what is likely to happen)

## 2. Business-driven component

- A threshold that determines when to act
- Ideally derived from expected value considerations
- Based on business costs, strategy, and domain knowledge

The model produces probabilities — not decisions.

There are multiple ways to set a threshold for classification, depending on the context.

A decision only emerges when probabilities are combined with a chosen threshold — this is where predictive analytics becomes prescriptive analytics.

# Outlook: Machine learning

# From statistical models to machine learning

Logistic regression is a classification model and represents a basic form of machine learning:

- learns from data
- makes predictions
- but with strong structural assumptions

It is:

- Interpretable
- Fast and robust
- Often a baseline model

Logistic regression is often the first model to try — simple, transparent, and surprisingly effective.

💡 More complex models are useful only if they improve performance meaningfully

# From statistical models to machine learning



## New challenges in machine learning

### 1. Overfitting

- Models may fit noise instead of signal
- Good training performance  $\neq$  good generalization

### 2. Role of data

- More observations and features become critical

### 3. Evaluation

- Same metrics (accuracy, precision, recall, F1)
- But evaluated on unseen data

### 4. Model tuning

- Not only thresholds
- Additional hyperparameters

| Logistic regression      | Machine learning models           |
|--------------------------|-----------------------------------|
| Linear decision boundary | Can learn nonlinear boundaries    |
| Strong assumptions       | Fewer assumptions                 |
| Selected variables       | Many variables (high-dimensional) |
| Interpretable            | Often less interpretable          |

### Key shift:

- From structured, theory-driven models
- To flexible, data-driven models

# Summary



- Logistic regression is a method for **binary classification**, where the goal is to predict whether an outcome occurs (e.g., churn vs. no churn).
- It predicts **probabilities**  $P(y = 1 \mid X)$  using a linear predictor combined with a **sigmoid function**, ensuring outputs between 0 and 1.
- Coefficients are interpreted in terms of **log-odds** (or odds ratios), not direct probability changes.
- Parameters are estimated via **maximum likelihood**, aiming to match predicted probabilities with observed outcomes.
- Model performance is evaluated using **classification metrics** such as confusion matrices, accuracy, precision, recall, F1, and ROC/AUC.
- A **threshold** is required to convert probabilities into class labels, and should be chosen based on **business costs**, not fixed at 0.5.

# Survey: Session 5



<https://forms.gle/mVWw3z7ftFn48gDY6>

Note: Responses may be analyzed and published in anonymized form.

Please complete the survey before you leave today — thank you 🙏

# References



Elkan, C. (2001). The foundations of cost-sensitive learning. *International Joint Conference on Artificial Intelligence*, 17, 973–978.  
<https://doi.org/10.5555/1642194.1642224>